Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA
www.dei.estg.ipleiria.pt

Systems Integration

Web Services – SOAP

Topics:

- Consume public web services
- Implement a WCF Service (SOAP), with .NET Core.
- Create, publish, and consume web-services using the development server
- Install and configure Web Server
- Publish web-services in a Web Server
- Consume web-services hosted by a Web Server
- Publish web-service in cloud services

Duration: 1 class

©2026-2027: {marisa.maximiano, nuno.costa, humberto.ferreira}@ulo.pt

- PART 1 -
ACCESSING AND CONSUMING PUBLIC WEB SERVICES

Create an **Avalonia (.NET)** project named **GlobalHolidaysClient** that consumes a public Web Service available in the cloud.

1. The web service contact is available at http://services.sapo.pt/Metadata/Contract/Holiday?culture=PT

1.1. Using the web service above, create an **Avalonia .NET App** that displays the public holidays for a specified year.

- Add a Service reference using the web service URL and the **dotnet-svcutil**.

Useful commands:

Install the tool[1]:

```
dotnet tool install --global dotnet-svcutil
```

Generates a new Service Reference:

```
dotnet-svcutil https://example.com/service.svc
```

Updates an existing service reference:

```
dotnet-svcutil -u ServiceReference
```

- Include the namespace generated by **dotnet-svcutil** at the beginning of the source code (e.g.: `using ServiceReference;`).
- Open the generated `Reference.cs` file inside the `ServiceReference` folder and compare the generated asynchronous methods with the information provided in the service contract.

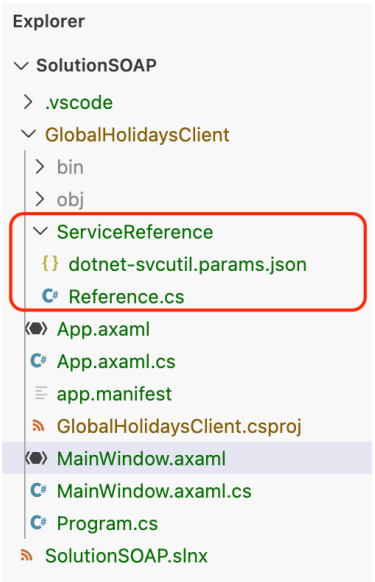


Figure 1 - Project files after adding the service reference

[1] If it was not installed at the beginning of the semester along with the required software.

1.2. Create the user interface shown in Figure 2 by adding the required controls to `Main-Window.axaml`.

1.2.1. Use a **NumericUpDown** control for year selection The default value should be the current year.

1.2.2. Use a **Listbox** to display holiday information. Examine the classes provided by the web service, determine which class contains the holiday details, and use that class to retrieve and present the holiday data.

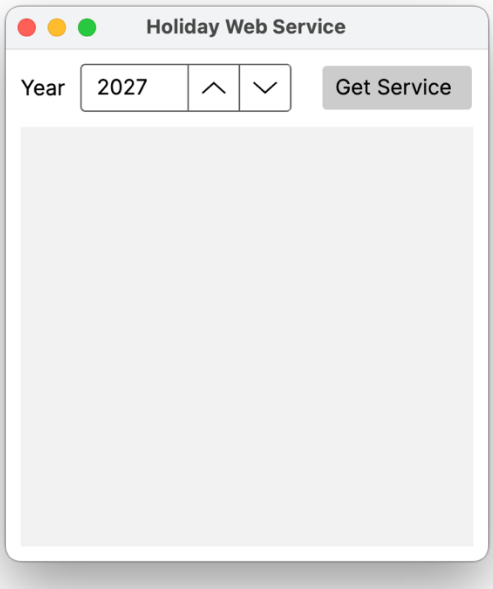


Figure 2 - Holiday web service client.

Proposed user interface:

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="450"
        x:Class="GlobalHolidaysClient.MainWindow"
        Width="320"
        Height="340"
        Title="Holiday Web Service">
    <Grid Margin="10"
          RowDefinitions="Auto,*">
        <!-- Painel superior -->
        <Grid ColumnDefinitions="Auto,140,20,Auto,*"
              Margin="0,0,0,10">
            <TextBlock Grid.Column="0"
                       Text="Year"
                       VerticalAlignment="Center"
```

```
                    Margin="0,0,10,0"/>
            <NumericUpDown Grid.Column="1"
                           Name="numericUpDownYear"
                           Minimum="1900"
                           Maximum="2100"
                           Increment="1"
                           Width="140"/>
            <Button Grid.Column="3"
                    Name="GetServiceButton"
                    Content="Get Service"
                    Width="100"
                    HorizontalAlignment="Left"
                    Click="GetServiceButton_Click"/>
        </Grid>
        <!-- Lista -->
        <ListBox Grid.Row="1"
                 Name="listBoxHoliday"/>
    </Grid>
</Window>
```

- PART 2 -
IMPLEMENTING A WEB SERVICE (WCF SOAP SERVICE) AND HOSTING IT

2. Create a new **CoreWCF Service** project named '**CoreWCFServiceBookstore.** We are going to implement a Web Service **ServiceBookstore.**

```
dotnet new corewcf -n CoreWCFServiceBookstore
cd WcfServiceBookstore
```

3. Run the web service and identify its service endpoint URL. Open the endpoint in a web browser and examine the information displayed. Verify that the service is running correctly and that the service metadata is accessible.

   Tip: The endpoint URL can usually be found in terminal when the service starts, or in the project's configuration files.

4. Open the `IService.cs` file and review the generated code. Rename the default "dummy" types as follows:

   - `IServiceBookstore` interface → Service contract
   - `ServiceBookstore` class → Service implementation
   - `Book` class → Book data contract (created by renaming the Composite-Type class).

4.1. The **Book** class is a **data contract** with the following data members: Title (string), Author (string), Year (int), Price (double) and Category (enumeration). Tip: You may move this class to a specific class file name Book.cs inside a folder named **Models.**

4.2. Create an enumeration (`enum`) named **BookCategory** containing the following values: WEB, CHILDREN, SCIENCE, ROMANCE, BIOGRAPHIES and OTHER. This enumeration must be used as the type of the **Category** property in the **Book** class. Tip: You may create the enumeration in a separate file named BookCategory.cs inside the Models folder.

4.3. Add the following service operations to the **IServiceBookstore** service contract, as shown in Figure **3**.

```
[ServiceContract]
7 references
public interface IServiceBookstore
{
    [OperationContract]
    2 references
    List<Book> GetBooks();

    [OperationContract]
    2 references
    Book? GetBookByTitle(string title);

    [OperationContract]
    2 references
    List<Book> GetBooksByCategory(BookCategory category);

    [OperationContract]
    2 references
    void AddBook(Book newBook);

    [OperationContract]
    2 references
    bool DeleteBook(string title);
}
```

Figure 3 - Suggested operation to be implemented in the web service.

4.4. Then, implement the corresponding operations in the **ServiceBookstore** service class. The operations must store and retrieve data from an XML file named `bookstore.xml`. You may reuse the `bookstore.xml` file created in the previous worksheet.

Before running the service, verify the following:

Because the service has been renamed, some references in the **`Program.cs`** file will no longer be valid. Update the file so that it uses the new service name and associated types.

Note: Only methods marked with the `[OperationContract]` attribute are exposed through the web service endpoint and are therefore available for testing.

4.5. Start the WCF service and ensure it is running correctly (see Figure 4). In the terminal window, locate and record the endpoint address displayed when the service starts. Open the endpoint address in a web browser and confirm that the service is available.

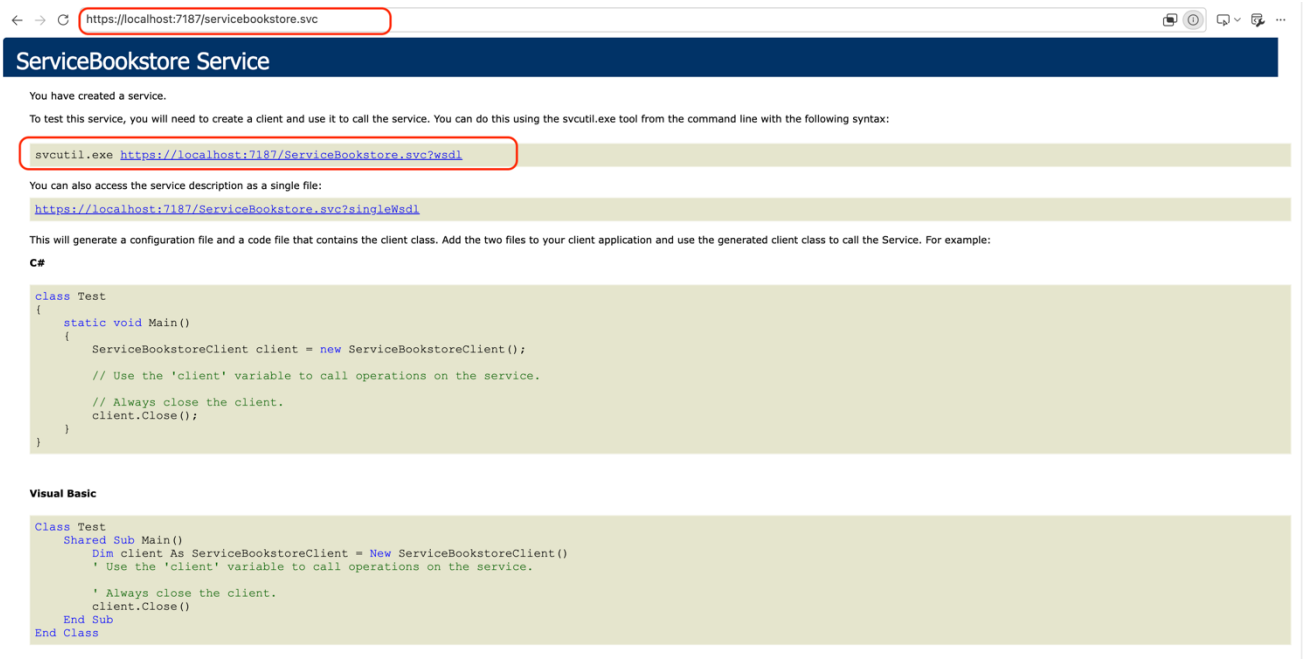


Figure 4 – Web service running in the browser.

4.6. [Optional] Use **SoapUI** to test the web service you have created.

4.6.1. Open **SoapUI** and create a new SOAP project using the service's WSDL URL.

4.6.2. Verify that the available operations are displayed.

Linux:

```
sudo apt install openjdk-17-jre

wget https://s3.amazonaws.com/downloads.eviware/soapuios/5.7.0/SoapUI-5.7.0-linux-bin.tar.gz

tar -xvzf SoapUI-5.7.0-linux-bin.tar.gz

cd SoapUI-5.7.0/bin

./soapui.sh
```

Windows / macOS:

Install Java 17 (or later) if not already installed.

Download the SoapUI Open Source installer for your OS from:
https://www.soapui.org/downloads/soapui/ (Windows: .exe installer; macOS: .dmg package) and run it.

Launch SoapUI from the Start Menu (Windows) or Applications folder (macOS).

- PART 3 –
CREATING A CLIENT APPLICATION TO CONSUME YOUR LOCAL WEB SERVICE

5. Create a new **Avalonia .NET App** project and name it '**ClientBookstore'**.

```
dotnet new avalonia.app -n ClientBookstore
cd ClientBookstore
```

Ensure that the project targets .NET 10 by setting the following value in the .csproj file:

```
<TargetFramework>net10.0</TargetFramework>
```

Restore the project dependencies and verify that the application builds successfully:

```
dotnet restore
dotnet build
```

6. Generate a client-side proxy for the `ServiceBookstore` SOAP service using **dotnet-svcutil**. Verify that the proxy classes and configuration files are added correctly to the project.

```
dotnet-svcutil http://localhost:5000/ServiceBookstore.svc
```

7. Implement a user interface similar to the one shown in the figure below (see Figure 5). The application must allow users to interact with the **ServiceBookstore** service by providing access to all available service operations.

   The application should support the following functionalities:

   - Retrieve and display all books.
   - Search for books by category.
   - Search for a book by title.
   - Add a new book.
   - Delete an existing book.
   - Display the results returned by the web service.

   Use the generated service proxy to invoke the SOAP service operations.

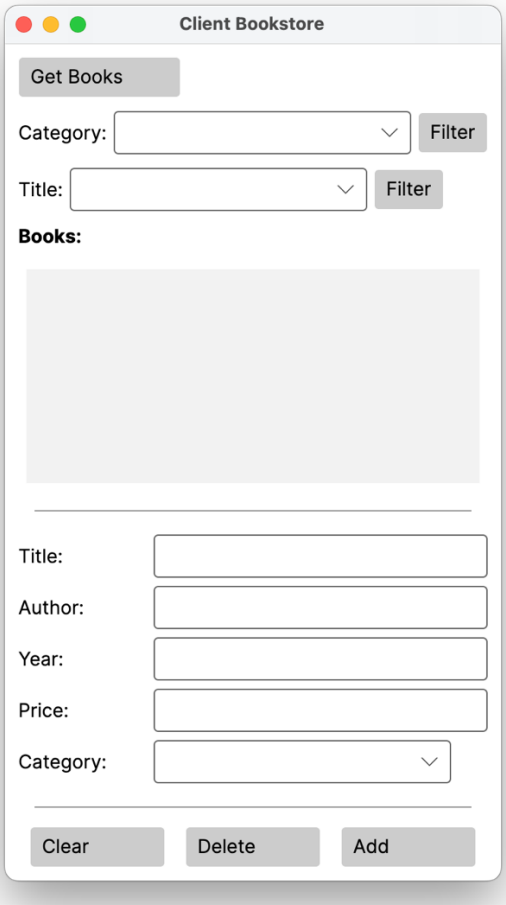


Figure 5 – Bookstore client application interface to test the web service.

7.1. **[Optional]** When using .NET to create SOAP web services that run on multiple operating systems, you have the opportunity to deploy them in many different ways, as they are no longer limited to Microsoft web servers. For example, try deploying your SOAP web service to a free Linux VPS.